

Note de transfert : modèles SCONTO-SVU

Version générique, variante DataCo/Prophet/Recalibrator, fondation expérimentale

Note intermédiaire V1, document de travail

1 Objet du document

Cette note décrit l'état des dépôts et modèles SCONTO-SVU au moment du transfert, pour permettre à un développeur qui rejoint le projet de comprendre rapidement quelles versions existent, laquelle est stable, laquelle est spécialisée DataCo, laquelle est expérimentale, ce qui a été corrigé, ce qui reste à faire, et ce qui peut ou non être fusionné à ce stade.

Cette note est une version intermédiaire (V1). Les expérimentations désignées EXP-0, E4 et E5 sont en cours ; aucune n'est présentée ici comme définitivement validée. La branche expérimentale ne doit pas être fusionnée dans le master stable avant les validations décrites section 14.

2 Cartographie des versions

Deux dépôts Git et un ensemble de variantes locales du modèle DataCo sont concernés.

Dépôt	Branche	Commit	Rôle
sconto-svu-generic-anylogic-model	integration/claude-generic-merge	ba05f34	Master générique stable, fusion figée
sconto-svu-generic-anylogic-model	feature/experimentation-foundation	e94f5ec	Fondation expérimentale (EXP-0), en validation
anylogic-model-documentation	documentation-latex-rewrite	3698c14	Documentation technique DataCo/Prophet/Recalibrator

Le modèle générique et la variante DataCo ne sont **pas** le même artefact : le noyau générique ne contient ni Prophet, ni Recalibrator, ni AutoCommande DataCo (section 4). Les variantes DataCo identifiées dans l'espace de travail local sont cartographiées à la section 5.

3 Version générique stable

Branche : integration/claude-generic-merge. **Commit** : ba05f348729ec607ae4716abf97db1f7b4a7cce6. **Artefact** : model/SCONTO_SVU_GENERIC_MASTER.alp.

Statut : fusion terminée, validée par build et runs utilisateur, figée avant toute expérimentation. Ce commit est le point de départ exact de la branche expérimentale décrite section 9 ; aucun changement n'y a été apporté depuis.

Les blocs suivants ont été intégrés et validés (build AnyLogic et runs utilisateur) au cours de la fusion :

- architecture générique pilotée par scénarios JSON (aucune configuration métier codée en dur dans le moteur) ;
- hiérarchie SCOR (processus Source, Make, Deliver, Return, niveaux P1 à P5) ;
- cartographie VSM des micro-activités et des flux ;
- architecture multi-agents holonique (ADACOR/PROSA) : agents stratégique, tactiques, coordinateurs, opérationnels ;

- hiérarchie ISA-95 et messagerie AER (tableau noir partagé);
- pipeline de Performance Index (méthode Theeranuphattana/Chan-Qi adaptée, agrégation floue) et indicateurs KPI par poste;
- politiques de stock MTS, MTO et ETO;
- gestion multi-commandes et multi-produit;
- réapprovisionnement autonome par produit (ordres REAPPRO_*);
- ordonnancement réel par insertion FIFO dans `actionsMetierDifferees` (section 7);
- exports Excel et ontologiques (ABox RDF/Turtle).

E4 et E5 (section 12) ne sont **pas** implémentées dans cette version : aucune variable de décision, aucun espace de configurations, aucune boucle Pareto/AHP de campagne d'optimisation n'existe dans ce commit.

4 Version DataCo / Prophet / Recalibrator

La variante DataCo est une extension spécialisée du modèle générique, maintenue comme un artefact distinct. Elle contient notamment :

- Prophet, prévision mensuelle de référence entraînée sur la période 2015 à 2016;
- l'AutoCommande DataCo, qui rejoue la demande 2017 sous forme de commandes clientes journalières;
- le Recalibrator, qui combine Prophet et la demande observée via un facteur de confiance $\alpha_j = \sqrt{j/N}$ (confirmé dans le code : `double alpha = Math.sqrt(Math.max(0.0, Math.min(1.0, (double) jour / (double) dernierJour)));`);
- le fonctionnement multi-produit DataCo (familles `DATA_CO_SPORTS`, `DATA_CO_CLOTHING`, `DATA_CO_ELECTRONICS`, configurées par scénario JSON, non codées en dur dans le moteur générique);
- les prévisions mensuelles et la recalibration hebdomadaire;
- les exports de résultats (Excel, ABox) associés aux campagnes DataCo.

Le modèle générique ne contient pas Prophet ni Recalibrator. Cette séparation est volontaire et documentée dans les instructions de fusion du dépôt : le noyau générique peut recevoir des capacités génériques utiles à DataCo (multi-produit générique, stocks par produit, interfaces d'extension), mais aucune connaissance métier DataCo codée en dur. Les fonctions `genererCommandeDataCoPourDate()`, `recalibrerPrevisionPourDate()` et les structures `prophetForecastsOriginal` sont absentes du master générique (ba05f34); elles n'existent que dans les variantes DataCo répertoriées section 5.

Ne pas injecter Prophet/Recalibrator dans le master générique pendant une opération de fusion sans décision architecturale explicite.

4.1 Résultats DataCo de référence

Les valeurs suivantes proviennent du run de référence RUN 1 (`simToRealSeconds = 3600`), documenté dans `documentation/assets/dataco_final_run/RUN_REFERENCE.md` du dépôt de documentation.

Indicateur	Valeur
Commandes générées	365
Demande totale	99 896
Commandes closes	79
dont closes en retard	20
Commandes EN_ATTENTE	286
Quantité livrée au snapshot	22 469
Performance Index au snapshot	8,871/10

Métrique	Prophet	Recalibrator
MAE	2 491,89	123,06
MAPE	45,75 %	1,91 %
RMSE	3 441,15	180,71

Réduction de l'erreur absolue totale entre Prophet et Recalibrator : environ 95,06 %.

RUN 1 n'est pas terminal. 286 commandes restent EN_ATTENTE au snapshot ; le Performance Index de 8,871/10 est un indicateur intermédiaire, pas une performance logistique terminale. Ces valeurs ne doivent pas être présentées comme un résultat final de la campagne.

5 Identification de la variante DataCo réellement exécutée

Onze fichiers `.alp` liés à DataCo/Prophet/Recalibrator ont été localisés dans l'espace de travail. Pour chacun : chemin, taille, SHA-256, présence des fonctions `genererCommandeDataCoPourDate()` et `recalibrerPrevisionPourDate()`, et statut.

Fichier	Taille (o)	Fonctions DataCo	Statut
sources/model/SCONTO_SVU_FINAL_VSM_FIX_CANDIDATE.alp	2 237 244	Absentes	Candidat VSM antérieur à l'intégration DataCo ; ne contient ni Prophet ni AutoCommande DataCo
..._DATACO_COMPLETE.alp (migration)	2 338 928	Présentes	Étape de développement DataCo (2026-09-09)
..._DATACO_AUTOMATED.alp (migration)	2 336 084	Présentes	Étape de développement (2026-09-09)
..._DATACO_AUTOMATED_CLOCK.alp (migration)	2 338 176	Présentes	Étape de développement (2026-09-09)
..._DATACO_DYNAMIC_GRAPHS.alp (migration)	2 334 561	Présentes	Étape de développement (2026-09-11)
..._DATACO_YEAR_BOUNDED_FINAL.alp (migration)	2 237 392	Présentes	Étape de développement (2026-09-11)
..._DATACO_BARCHARTS_REALISTIC_RECAL.alp (migration)	2 367 526	Présentes	Étape de développement (2026-09-11)
..._DATACO_MULTIPRODUCT_FOUNDATION.alp (migration)	2 376 717	Présentes	Étape de développement (2026-09-12)
..._DATACO_MULTIPRODUCT_CONCURRENT_FIX.alp (migration, 2026-09-15)	2 371 853	Présentes	Une seule différence identifiée avec SCONTO_SVU_DATACO_FINAL.alp : le champ interne <Name>, voir ci-dessous
reference/dataco-multiproduct/..._DATACO_MULTIPRODUCT_CONCURRENT_FIX.alp (dépôt générique, 2026-09-16)	2 244 316	Présentes	Même nom que la ligne précédente, contenu substantiellement différent (voir remarque)
SCONTO_SVU_LIVRAISON_ENCADREMENT/01_MODELES/SCONTO_SVU_DATACO_FINAL.alp	2 371 806	Présentes	Fichier co-localisé avec les exports RUN 1/RUN 2 ; retenu comme modèle de référence de la campagne DataCo documentée

5.1 Résultat de la comparaison

La comparaison octet par octet entre SCONTO_SVU_DATACO_FINAL.alp et la variante ..._DATACO_MULTIPRODUCT_CONCURRENT_FIX.alp du dossier de migration (datée du 2026-09-15) montre une différence unique : le champ interne <Name> du projet AnyLogic (SCONTO_SVU_DATACO_FINAL contre SCONTO_SVU_FINAL_VALIDATED_FORECAST_DATACO_MULTIPRODUCT_CONCURRENT_FIX). Après normalisation de ce seul champ, les deux fichiers sont identiques. **Conclusion démontrée, non supposée** : SCONTO_SVU_DATACO_FINAL.alp est une copie renommée de cette variante, sans aucune autre modification.

Les deux fichiers conservent néanmoins des empreintes SHA-256 distinctes, précisément parce que le champ <Name> diffère : un hash cryptographique change dès qu'un seul octet du fichier change, y compris pour un champ de métadonnées sans effet fonctionnel. L'identité fonctionnelle

démontrée ci-dessus repose sur la comparaison de contenu après normalisation de ce champ, pas sur une égalité des empreintes brutes.

Le fichier de même nom présent dans `reference/dataco-multiproduct/` du dépôt générique, daté du lendemain (2026-09-16), est en revanche substantiellement différent (taille inférieure d'environ 127 Ko, différences réparties sur l'ensemble du fichier) : il s'agit d'un artefact distinct malgré le nom identique. Cette homonymie est une source de confusion à corriger (renommage ou suppression d'une des deux copies) avant toute reprise du projet.

Le **champ runtime modeleCandidate**, présent dans le manifeste exporté par les campagnes DataCo, est une étiquette codée en dur dans le générateur de manifeste (confirmée identique dans plusieurs variantes du dépôt) et **ne constitue pas une preuve** du fichier réellement exécuté ; il ne doit pas être utilisé comme tel. L'identification ci-dessus repose exclusivement sur la comparaison de contenu et sur la co-localisation du fichier avec les exports RUN 1/RUN 2.

6 Principales évolutions du modèle

6.1 Ligne générique

Fusion progressive de `model/SCONTO_SVU_GENERIC_MASTER.alp` avec les apports du modèle collègue, en six blocs fonctionnels (A, M, B, C, D, E), chacun audité, corrigé, validé statiquement puis par build et run utilisateur avant de passer au suivant. Clôture du dernier bloc (E, ordonnancement) et gel du master au commit `ba05f34`.

6.2 Ligne DataCo

Progression documentée par une suite de fichiers `.alp` successifs (2026-09-09 à 2026-09-15) : intégration initiale, automatisation de l'horloge, graphiques dynamiques, bornage calendaire de l'année cible, recalibration réaliste, fondation multi-produit, puis correction de concurrence multi-produit, ce dernier état étant celui effectivement exécuté pour produire RUN 1/RUN 2 (section 5).

6.3 Fondation expérimentale

Trois passes successives sur la branche `feature/experimentation-foundation`, détaillées section 9.

7 Corrections fonctionnelles importantes

Problème	Correction	Effet
Commandes MTO/ETO client bloquées en boucle infinie <code>ANALYSE_STOCK</code> (garde de sécurité qui ne couvrait pas ces types)	Séparation stricte MTS / MTO-ETO dans <code>analyserStockCommandeOrchestree()</code> : après les deux branches MTS inchangées, appel direct à <code>analyserMatiereCommandeOrchestree()</code> , déjà utilisée sans modification par le réapprovisionnement autonome	Les commandes MTO/ETO progressent jusqu'à clôture; garde de réentrance étendue pour éviter une double production en cas de réveil concurrent
Ordonnanceur dédié (<code>ordonnancerProduction()</code> , tri priorité/EDD) jamais exécuté utilement	Audit et confirmé inactif : <code>carnetCommandesMTO</code> n'est jamais peuplé dans tout le master (0 appel <code>.add()</code>); clarifié comme mécanisme hérité non actif, non réactivé	La politique réelle d'ordonnancement, FIFO par insertion dans <code>actionsMetierDifferees</code> , est documentée comme telle plutôt que supposée
Association flux/commande imprécise (<code>CMD_4</code> pouvait matcher <code>CMD_44</code>)	Matching strict de préfixe	Aucune confusion entre commandes aux identifiants proches
Engagement client (délai promis <code>tPromise</code>) écrit deux fois, dont un calcul mort avant résolution complète du scénario	Écriture prématurée supprimée; seule écriture conservée, après résolution du scénario et du type de commande; délai par défaut et par produit rendus configurables (<code>champDelaiPromesseParDefautSec</code> , <code>ScenarioFlux.promisedDelaySec</code>)	Une seule source de vérité pour <code>tPromise</code> , configurable sans modification du moteur
Détection des retards clients non centralisée	<code>commandeEstTerminale()</code> établi comme prédicat canonique unique (<code>SERVIE/EN_RETARD/REFUSEE/BLOQUE_*</code>); <code>retardConstate</code> et <code>modeEngagementClient</code> comme champs dédiés, détection continue via événement périodique	Une seule définition de la terminalité et du retard, réutilisée par tous les compteurs (KPI, exports, diagnostics)
Compteur de commandes ouvertes potentiellement ambigu (client vs réapprovisionnement)	Vérifié par lecture directe : exclut déjà les ordres <code>REAPPRO_*</code> via <code>estOrdreStockAutonome()</code>	Compteur <code>nombreCommandesOuvertes()</code> correct pour un usage « commandes clientes uniquement » sans modification nécessaire
Incohérences RL/RS du pipeline SCOR vers PI (proxies non calibrés, métrique retournant 0 malgré stock non nul, warm-up affichant PI=0)	Corrections ciblées bloc par bloc (B.1 à B.3-FIX2), sources et unités alignées, proxies explicitement documentés comme tels	Pipeline PI/SCOR revallidé par build et runs utilisateur, clôturé et figé

8 Architecture SCOR / VSM / SMA conservée

L'architecture retenue à l'issue de la fusion reste inchangée par les travaux EXP-0 :

- **SCOR** : processus Source, Make, Deliver, Return, codes de niveau 1 à 3, catalogue de métriques associé à chaque code;
- **VSM** : micro-activités et flux physiques/informationnels cartographiés par scénario JSON;

- **SMA holonique (ADACOR/PROSA)** : un agent stratégique (**StrategicAgent**), quatre agents tactiques (un par macro-processus SCOR principal, **TacticalAgent**), des coordinateurs dédiés par macro-processus (**CoordinatorAgent**) et des agents opérationnels par poste (**OperationalAgent**);
- **ISA-95** : hiérarchie industrielle conservée dans l'export ontologique;
- **AER** : messagerie d'alerte/escalade via un tableau noir partagé (**BlackboardAgent**), remontée opérationnel → tactique → supervision;
- **Performance Index** : méthode Theeranuphattana/Chan-Qi adaptée, agrégation floue des cinq attributs (RL, RS, AG, CO, AM).

9 Expérimentations et instrumentation

Branche `feature/experimentation-foundation`, créée à partir de `ba05f34`. Trois commits, chacun documenté dans `EXPERIMENTATION_FOUNDATION.md` :

- **56f763f**, fondation initiale : audit exhaustif des sources d'aléatoire (toutes passent par `getDefaultRandomGenerator()`); paramètre `seedExperiment` appliqué en tête de `demarrerSimulation()`; distinction entre fin de génération de la demande et état terminal opérationnel; diagnostic terminal structuré; champs de traçabilité additifs dans l'export ontologique.
- **82f60d2**, vérification de trois ambiguïtés : confirmation que le compteur de commandes ouvertes exclut déjà les ordres de réapprovisionnement; confirmation que la définition de fin de génération couvre le scénario de choc de demande utilisé comme référence pour une future campagne d'optimisation; correction de la persistance de la graine dans le scénario JSON (sérialisation en chaîne plutôt qu'en nombre à virgule flottante, pour éviter une perte de précision).
- **e94f5ec**, correction d'un défaut de reproductibilité réel, mis en évidence par un test runtime à graine identique : plusieurs événements métier cycliques étaient phasés sur le temps absolu du moteur AnyLogic plutôt que sur le début logique du run. Huit événements rephasés via l'API standard `restart()`, sans modification d'aucune règle métier, période ou paramètre de panne.

Statut : en cours de validation runtime. Ne pas fusionner dans le master stable à ce stade.

10 Reproductibilité

Le test runtime effectué après le commit `56f763f` a montré que, même avec `seedExperiment=1001` identique sur deux runs, les trajectoires divergeaient à partir du traitement d'un ordre de réapprovisionnement autonome. La cause exacte, confirmée par lecture du code, est un événement cyclique (`cycleArbitrage`, agent tactique) configuré pour se déclencher sur une grille de temps absolue du moteur, indépendamment de l'instant où l'utilisateur démarre la campagne.

La correction appliquée au commit `e94f5ec` rephase huit événements métier identifiés comme pertinents (mutation d'état ou décision affectant le flux) relativement au début logique du run.

Point de vigilance explicite : cette correction n'a pas encore été validée par un nouveau test runtime à deux graines identiques après `e94f5ec`. Statut exact : **implémentée, build à valider, runtime à revalider.**

11 Travaux encore en cours

- Revalidation runtime de la correction de rephasage (section précédente) : deux runs à `seedExperiment` identique doivent produire des trajectoires identiques après normalisation

au début du run.

- Un état latent distinct a été identifié sans être corrigé : `tProchainePanne` et `enPanne` (par poste) ne sont pas explicitement réinitialisés entre plusieurs runs successifs dans une même session AnyLogic. Non observé dans le test effectué, probablement parce que la valeur initiale restait à son défaut avant le premier run ; à surveiller si une session enchaîne plusieurs runs sans rouvrir le modèle.
- Le problème `demandGenerationFinished` observé à la clôture finale du run (`RUN_FINALIZED`) a été signalé par l'utilisateur mais n'a pas encore été investigué : aucune dépendance technique avec la correction de rephasage n'a été démontrée, traitement reporté à une passe distincte.
- E4 et E5 (section 12) ne sont pas exécutées.

12 E4 et E5, travaux futurs

Aucune de ces campagnes n'est exécutée à ce jour. Elles sont présentées ici uniquement comme protocoles formalisés dans la documentation, à titre de travaux futurs s'appuyant sur la fondation expérimentale EXP-0.

E4, optimisation ZENER. Variables de décision : x_1 (capacité du poste critique), x_2 (taille des sous-lots), x_3 (règle de priorité), x_4 (politique de stock), soit 144 configurations candidates. Méthode : exploration factorielle, front de Pareto, puis sélection AHP parmi les solutions Pareto-efficaces.

E5, optimisation DataCo fondée sur la prévision. Compare quatre politiques : P0 (réactive, sans prévision), P1 (Prophet seul), P2 (Prophet et Recalibrator), P3 (Prophet, Recalibrator et paramètres de planification optimisés). Variables de décision par famille simulée : $x_p = (SS_p, s_p, Q_p, f_p)$.

13 État actuel des branches Git

Dépôt / branche	HEAD	Statut
sconto-svu-generic-anylogic-model integration/ claude-generic-merge	ba05f34	Stable, figée, fusion terminée
sconto-svu-generic-anylogic-model feature/ experimentation-foundation	e94f5ec	Expérimentale, en validation runtime, non fusionnée
anylogic-model-documentation documentation-latex-rewrite	a10652d	HEAD après ajout de la présente note ; documents scientifiques DataCo/Prophet/Recalibrator à jour depuis le commit 3698c14

14 Instructions de merge pour le collègue

A. Pour récupérer le modèle générique stable : checkout `integration/claude-generic-merge` au commit `ba05f34`. Artefact : `model/SCONTO_SVU_GENERIC_MASTER.alp`.

B. Ne pas fusionner `feature/experimentation-foundation` pour l'instant : le runtime est encore en cours de validation (section 9).

C. Conserver la variante DataCo séparée du master générique.

D. Ne pas injecter Prophet ni Recalibrator dans le master générique pendant une opération de fusion sans décision architecturale explicite.

E. Avant tout merge futur de la branche expérimentale, valider dans l'ordre :

1. build AnyLogic 8.9.5 sans erreur ;
2. reproductibilité à `seedExperiment=1001` (deux runs, mêmes trajectoires après normalisation) ;
3. divergence attendue à `seedExperiment=1002` ;
4. les trois cas de terminalité (génération en cours, génération terminée avec encours, terminal atteint) ;
5. réinitialisation de l'état des pannes entre runs successifs ;
6. non-régression des scénarios MTS, MTO, ETO, multi-commandes, multi-produit déjà validés sur le master.

15 Points de vigilance

- Deux fichiers portent le même nom (`..._DATACO_MULTIPRODUCT_CONCURRENT_FIX.alp`) dans deux emplacements différents, avec un contenu substantiellement différent (section 5) : à clarifier avant toute reprise.
- Le champ runtime `modeleCandidate` ne doit jamais être utilisé comme preuve d'identité d'un fichier exécuté.
- Les exports runtime volumineux (ABox TTL jusqu'à plusieurs Mo) ne sont pas committés dans le dépôt de documentation ; ils restent disponibles dans le dossier de livraison local et sont nommés dans `RUN_REFERENCE.md`.
- Le champ exporté `entitiesInProcessAtPosts` (EXP-0) compte des occupations dans les structures de traitement des postes, pas garanti comme un nombre d'agents métier uniques.
- Le modèle utilise un seul flux aléatoire partagé pour tous les mécanismes stochastiques : deux configurations qui consomment un nombre différent de tirages avant un point donné désynchronisent le flux au-delà de ce point, même à graine identique. Une séparation en générateurs dédiés par mécanisme resterait à faire pour un appariement strict des nombres aléatoires communs, si les campagnes E4/E5 le requièrent.

16 Tableau de traçabilité des artefacts

Artefact	Branche/dépôt	Commit	Rôle	Merge	Remarques
SCONTO_SVU_GENERIC_MASTER.alp	integration/claude-generic-merge	ba05f34	Master générique	Oui	Stable, figé
SCONTO_SVU_DATACO_FINAL.alp	hors dépôt structuré (livraison locale)	n/d	Modèle DataCo exécuté pour RUN 1/RUN 2	Non applicable	Identique après normalisation du champ <Name> à la variante migration 2026-09-15
..._VSM_FIX_CANDIDATE.alp	hors dépôt structuré	n/d	Candidat antérieur, sans DataCo	Non	Historique
feature/experimentation-foundation (branche)	sconto-svu-generic	e94f5ec	Fondation EXP-0	Non, en validation	Revalidation runtime requise
Documentation_Module_Prevision_SCONTO_SVU_corrige.tex	anylogic-model-documentation	3698c14	Documentation module DataCo	n/a	À jour
REPONSES_ENCADREMENT_PROPHET_ALPHA_OPTIMISATION.tex	anylogic-model-documentation	3698c14	Synthèse académique Prophet/ α_j /optimisation	n/a	À jour
RUN_REFERENCE.md	anylogic-model-documentation	3698c14	Traçabilité RUN 1/RUN 2	n/a	À jour

17 Recommandations pour la reprise par le collègue

1. Partir du master générique (ba05f34) pour toute nouvelle fonctionnalité générique ; ne pas repartir de la branche expérimentale tant qu'elle n'est pas revalidée.
2. Ne pas modifier `model/SCONTO_SVU_GENERIC_MASTER.alp` directement sans créer une branche dédiée, conformément au workflow déjà établi dans le dépôt.
3. Pour toute reprise du travail DataCo, utiliser `SCONTO_SVU_DATACO_FINAL.alp` comme référence, en gardant à l'esprit qu'il s'agit d'une copie renommée de la variante de migration datée du 2026-09-15, et non d'un artefact distinct.
4. Avant de fusionner `feature/experimentation-foundation`, exécuter le protocole de validation de la section 14.E dans son intégralité.
5. Clarifier l'homonymie signalée section 5 avant toute automatisation qui se fierait au nom de fichier plutôt qu'à son contenu.
6. Ne pas exécuter E4 ou E5 avant que la reproductibilité et la condition terminale de EXP-0 soient entièrement validées.

Annexe : chronologie des principales évolutions

- Fusion générique, blocs A (corrections sûres) et M (fondation multi-produit).

- Bloc B : refonte du pipeline Performance Index / SCOR.
- Bloc C : détection continue des retards, engagement client configurable.
- Bloc D : audit du moteur d'exécution et des exports.
- Bloc E : audit et correction de l'ordonnancement, raccordement du chemin de production MTO/ETO.
- Clôture et gel du master générique (ba05f34).
- Développement DataCo : Prophet, AutoCommande, Recalibrator, multi-produit DataCo (progression de fichiers `.alp` successifs, 2026-09-09 à 2026-09-15).
- Documentation technique DataCo/Prophet/Recalibrator (dépôt de documentation, plusieurs passes de rigueur scientifique).
- EXP-0 : graine expérimentale, condition terminale, diagnostic.
- EXP-0.1 : vérification de trois ambiguïtés (commandes ouvertes, couverture E4, précision du round-trip JSON de la graine).
- EXP-0.2 : correction de la reproductibilité runtime (rephasage des événements métier cycliques).